

# WaveVis inverse sheet model for a localized breaking-wave overhang

Alan N. Pham

AO Labs, WaveVis simulator, inverse deformation sheet paper

**This paper explains the inverse-sheet tab in WaveVis. The tab is a client-side design tool for comparing how a rectangular sheet can be mapped toward one localized plunging-wave shape while preserving a visible grid, boundary anchoring, deformation metrics, and failure blockers. It is not a finite-element solver, not a measured prototype report, and not a claim that the current grid can fabricate the reference wave. Its job is narrower: define the target deformation, show where strain, bend, shear, area change, planarity error, and dihedral rotation concentrate, and keep the known geometric failure classes visible before physical mechanism work starts.**

## 1 Introduction

The inverse-sheet problem starts from a flat rectangular sheet, not from a free decorative surface. The object in the app is a sampled sheet with a rest grid, a current grid, edges, quadrilateral cells, neighbor bending measures, and a localized target deformation. The user-facing question is physical: if a sheet must stay connected and its boundary must remain a reference frame, where does a wave-like overhang require large stretch, bend, shear, planarity error, or local discontinuity?

This question sits between geometry processing, soft-mechanism design, and computational inverse design. WaveVis borrows the idea that a target shape can be treated as an optimization or projection target from deformable simulation and design tools (1–6). It also borrows the design discipline of surface-based inflatables and architected sheets: the sheet geometry is not the result by itself; the result is the mismatch between the desired shape and the mechanism constraints that might realize it (7–9). The same caution applies to mechanical metamaterials and origami-inspired sheets: a pattern can look like a target while violating the real hinge, fold, or material rule that makes it physical (10–13).

The inverse-sheet tab therefore uses an explicit lightweight model. It does not hide the grid behind a smooth skin. It shows the sheet points, edges, surface, selected elements, local worst elements, and

27 scalar metrics. When the visible shape is bad, the model should make the badness inspectable. The  
28 accepted design loop is:

- 29 1. define the desired wave field in coordinates;
- 30 2. map rest nodes into target nodes;
- 31 3. preserve boundary nodes and support zones;
- 32 4. smooth only where smoothing is a stated display/model operation;
- 33 5. compute local geometric metrics;
- 34 6. reject shapes with known impossible or misleading features;
- 35 7. use the metrics to decide the next mechanism or topology test.

36 Older iterations failed when the paper or app treated a pretty silhouette as evidence. A profile trace is  
37 not a sheet. A surface skin is not a mechanism. A smooth overhang image is not a proof that the grid  
38 can reach it. The present inverse-sheet paper is written to prevent that failure from recurring.

## 39 **2 Source Map**

---

40 The active implementation is in the WaveVis source tree. The main source files for this paper are:

| Source file                                         | Role in this paper                                                                                                                               |
|-----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>src/latticeGeometry.ts</code>                 | Builds the inverse-sheet grid, target field, support field, smoothing passes, cavity blockers, edges, quads, dihedral pairs, and scalar metrics. |
| <code>src/inverseSheetTypes.ts</code>               | Defines the sheet configuration, nodes, edges, quads, metrics, bounds, selected elements, and camera/view state types.                           |
| <code>src/InverseSheetTab.tsx</code>                | Connects the control panel, model builder, export buttons, worst-element panel, and 3D viewer.                                                   |
| <code>src/LatticeViewer3D.tsx</code>                | Renders nodes, edges, surface, selected elements, heatmaps, mechanism overlays, and camera views.                                                |
| <code>scripts/<br/>check-inverse-sheet.cjs</code>   | Runs geometry and regression checks for the inverse-sheet surface and known blocker classes.                                                     |
| <code>scripts/<br/>readable-wave-metrics.cjs</code> | Measures whether generated wave candidates remain readable and rejects known visual failure families.                                            |

**Table 1.** Implementation files that define the inverse-sheet model. The paper describes these files; it does not replace them as source of truth.

### 3 Geometry

### 3.1 Rest grid

The sheet is a rectangular grid of nodes. With  $R$  rows and  $C$  columns, the node index set is

$$\mathcal{V} = \{(r, c) \mid 0 \leq r < R, 0 \leq c < C\}.$$

The current default in source is  $R = 44$ ,  $C = 112$ , and unit spacing in the canonical sheet coordinates.

The rest position of node  $i = (r, c)$  is

$$\mathbf{x}_i^0 = \begin{bmatrix} x_i^0 \\ y_i^0 \\ 0 \end{bmatrix} \in \mathbb{R}^3.$$

The grid stores three positions for every node: rest  $\mathbf{x}_i^0$ , target  $\mathbf{x}_i^*$ , and current  $\mathbf{x}_i$ . The current position is not solved by a physical time integrator; it is a morph interpolation,

$$\mathbf{x}_i = (1 - m)\mathbf{x}_i^0 + m\mathbf{x}_i^*,$$

except that the scalar  $m$  is eased by the source function `morphGrowthAmount`. This makes the interaction readable, but it is not a dynamic simulation.

### 3.2 Edges, quads, and dihedral pairs

The edge set has horizontal and vertical grid edges:

$$\mathcal{E}_h = \{((r, c), (r, c + 1))\}, \quad \mathcal{E}_v = \{((r, c), (r + 1, c))\}.$$

The total edge count is

$$|\mathcal{E}| = R(C - 1) + (R - 1)C.$$

Quadrilaterals are formed by every adjacent  $2 \times 2$  node block:

$$q_{r,c} = \{(r, c), (r, c + 1), (r + 1, c), (r + 1, c + 1)\},$$

with count  $(R - 1)(C - 1)$ . Dihedral pairs are formed between horizontally and vertically adjacent quads. They are only a geometric proxy for bending; no plate stiffness or material thickness is solved.

## 4 Target Deformation Field

56

### 4.1 Canonical frame

57

The target shape is defined in a canonical wave frame. A rest point  $\mathbf{x}^0$  is mapped to a canonical sample  $\hat{\mathbf{x}}$  by a steer/yaw transform and by the fixed sheet coordinate convention in source:

59

$$\hat{\mathbf{x}} = T_{\text{canonical}}(\mathbf{x}^0; \theta_{\text{steer}}).$$

Only points inside the canonical sheet and support field receive a target displacement. Outside that field,

60

61

$$\mathbf{x}^* = \mathbf{x}^0.$$

That rule is important. The app is not trying to curl the whole rectangular sheet. It is trying to localize a breaking-wave event while leaving a surrounding reference sheet.

62

63

### 4.2 Normalized coordinates

64

Inside the active field, the target uses normalized coordinates

65

$$u = \frac{\hat{x} - x_{\min}}{x_{\max} - x_{\min}}, \quad v = \frac{\hat{y} + s/2}{s},$$

where  $s$  is the field span. The deformation function can be written as

66

$$\mathbf{x}^* = \mathbf{x}^0 + R_{\text{yaw}}^{-1} \Delta(u, v; \mathbf{p}),$$

where  $\mathbf{p}$  is the sanitized configuration: height, horizontal offset, overhang width, overhang position, overhang angle, profile scale, curl radius, smoothing, wall smoothness, flat contribution, and display toggles.

67

68

69

The source implementation separates the target computation into several named operations:

70

$$\Delta_{\text{raw}} \rightarrow \Delta_{\text{span}} \rightarrow \Delta_{\text{flat}} \rightarrow \Delta_{\text{blocked}}.$$

### 4.3 Boundary rule

Every boundary node is pinned:

$$\mathbf{x}_i^* = \mathbf{x}_i^0 \quad \text{for } r \in \{0, R-1\} \text{ or } c \in \{0, C-1\}.$$

This is not a fabrication guarantee. It is a guardrail that prevents the target field from moving the whole sheet and pretending the shape is easy because the frame moved with it.

## 5 Smoothing and Support Operations

### 5.1 Span continuity smoothing

For generated profiles, the source runs `applySpanContinuitySmoothing`. The input is a grid of raw displacement vectors

$$\Delta_{r,c}^0 = \mathbf{x}_{r,c}^* - \mathbf{x}_{r,c}^0.$$

For interior nodes, the pass blends a node's displacement with row and column averages:

$$\Delta_{r,c}^{k+1} = (1 - \alpha_x - \alpha_y) \Delta_{r,c}^k + \alpha_x \frac{\Delta_{r-1,c}^k + \Delta_{r+1,c}^k}{2} + \alpha_y \frac{\Delta_{r,c-1}^k + \Delta_{r,c+1}^k}{2},$$

with local preserve weights near the centerline and terminal curl. The actual source uses separate weights for core preservation, terminal curl lock, row blending, and longitudinal blending. The mathematical meaning is simple: make the span continuous without erasing the localized curl.

### 5.2 Flat contribution diffusion

The flat contribution pass spreads part of the deformation into a surrounding apron. It computes pin weights  $w_i \in [0, 1]$  and participation weights  $a_i \in [0, 1]$ . A simplified pass is

$$\Delta_i^{k+1} = (1 - w_i) \left[ (1 - \alpha a_i) \Delta_i^k + \alpha a_i \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} \Delta_j^k \right] + w_i \Delta_i^0.$$

The pin term keeps core features from being washed away. The participation term lets the surrounding sheet absorb some displacement so the target does not read as a narrow strip pasted onto a flat plate.

### 5.3 Bowl and pocket blocker

The source function `blockSurfaceBowlPockets` rejects a class of target fields that look like a hidden inward pocket below the curl. For each active interior node, the pass compares its height with the minimum height of its four direct neighbors. If a node drops below the neighbor floor in an active region, it is lifted:

$$z_i \leftarrow \max(0, \min_{j \in \mathcal{N}_4(i)} z_j).$$

This is not a physical contact solve. It is a source-level blocker for a rejected visual artifact: dark pockets, bowls, dimples, and under-overhang support necks that previously made Alan do visual QA by hand.

## 6 Metrics

### 6.1 Edge strain

For edge  $e = (i, j)$ , the rest and current lengths are

$$L_e^0 = \|\mathbf{x}_j^0 - \mathbf{x}_i^0\|, \quad L_e = \|\mathbf{x}_j - \mathbf{x}_i\|.$$

The displayed strain is

$$\epsilon_e = \frac{L_e - L_e^0}{L_e^0}.$$

Positive values are tensile stretch; negative values are compression. The inverse-sheet tab reports max tensile strain, max compressive strain, mean absolute strain, and RMS strain. These values are geometric strain proxies, not material stress.

### 6.2 Edge rotation

The edge rotation angle is

$$\theta_e = \cos^{-1} \left( \frac{(\mathbf{x}_j^0 - \mathbf{x}_i^0) \cdot (\mathbf{x}_j - \mathbf{x}_i)}{\|\mathbf{x}_j^0 - \mathbf{x}_i^0\| \|\mathbf{x}_j - \mathbf{x}_i\|} \right).$$

Large rotation means an edge has moved far from its rest direction. It does not by itself mean the edge is mechanically invalid, but it is useful for locating the deformation front.

### 6.3 Node bend and shear

For an interior node, row bend is the deviation from a straight row:

$$b_i^{\text{row}} = |180^\circ - \angle(\mathbf{x}_{i-\hat{x}} - \mathbf{x}_i, \mathbf{x}_{i+\hat{x}} - \mathbf{x}_i)|.$$

Column bend is computed the same way in the y direction. The reported node bend is the larger of the two:

$$b_i = \max(b_i^{\text{row}}, b_i^{\text{col}}).$$

Shear is the deviation from orthogonality between local row and column directions:

$$s_i = |90^\circ - \angle(\mathbf{x}_{i+\hat{x}} - \mathbf{x}_i, \mathbf{x}_{i+\hat{y}} - \mathbf{x}_i)|.$$

### 6.4 Quad area, normal rotation, and planarity

For a quad  $q$ , the area is the sum of two triangle areas:

$$A_q = \frac{1}{2} \|(\mathbf{x}_{10} - \mathbf{x}_{00}) \times (\mathbf{x}_{01} - \mathbf{x}_{00})\| + \frac{1}{2} \|(\mathbf{x}_{11} - \mathbf{x}_{10}) \times (\mathbf{x}_{01} - \mathbf{x}_{10})\|.$$

The area change is  $A_q/A_q^0 - 1$ . Normal rotation is the angle between averaged rest and current quad normals. Planarity error is the distance of the fourth point from the plane through the other three:

$$p_q = \frac{|(\mathbf{x}_{11} - \mathbf{x}_{00}) \cdot ((\mathbf{x}_{10} - \mathbf{x}_{00}) \times (\mathbf{x}_{01} - \mathbf{x}_{00}))|}{\|(\mathbf{x}_{10} - \mathbf{x}_{00}) \times (\mathbf{x}_{01} - \mathbf{x}_{00})\|}.$$



## 6.5 Combined cost

116

The tab reports a scalar combined cost

117

$$J = w_\epsilon |\overline{\epsilon}| + w_b \overline{b} + w_s \overline{s} + w_d \overline{d},$$

where  $d$  is dihedral rotation. This is a ranking convenience. It is not a calibrated elastic energy. If

118

fabrication becomes the next step, the cost must be replaced or supplemented by material tests, plate

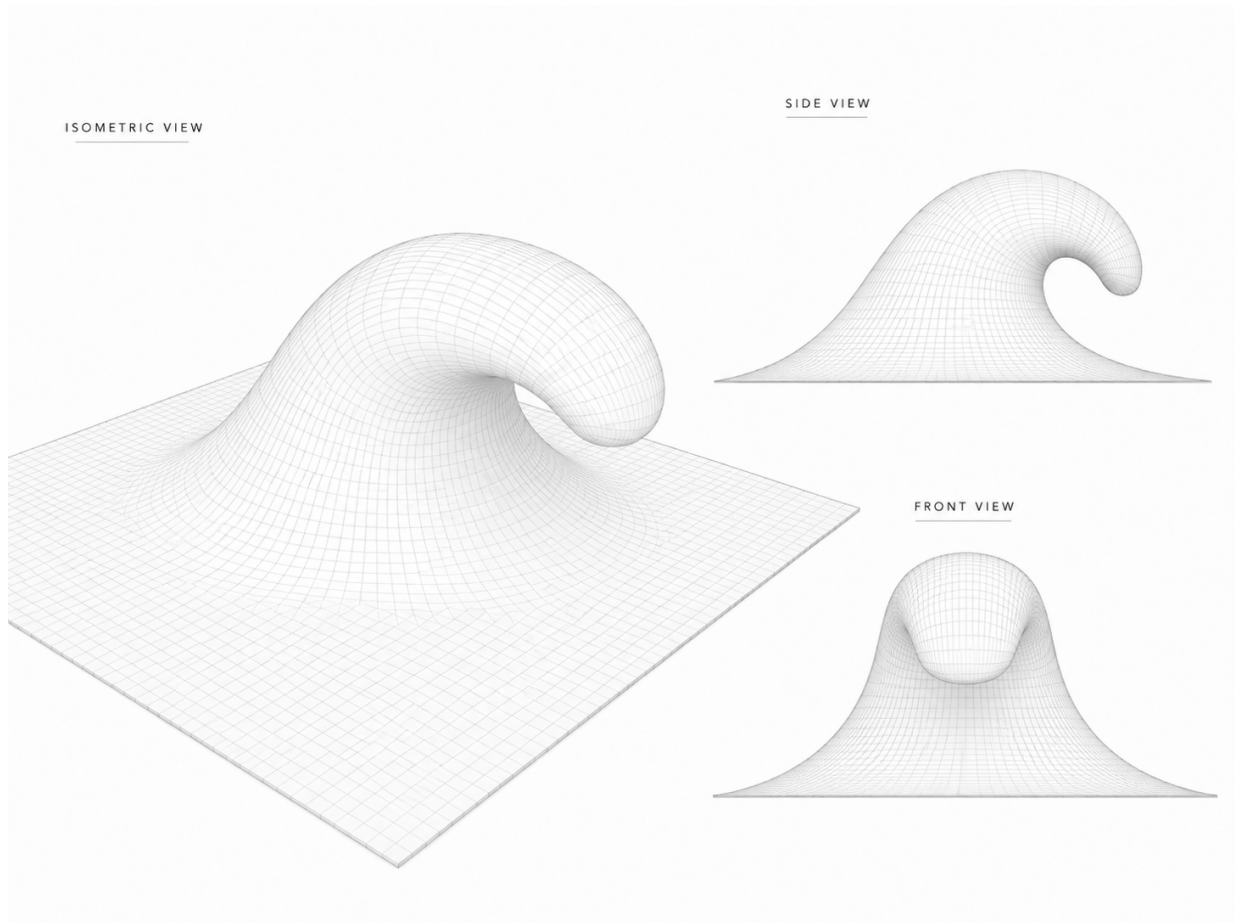
119

thickness, hinge stiffness, actuator geometry, and contact.

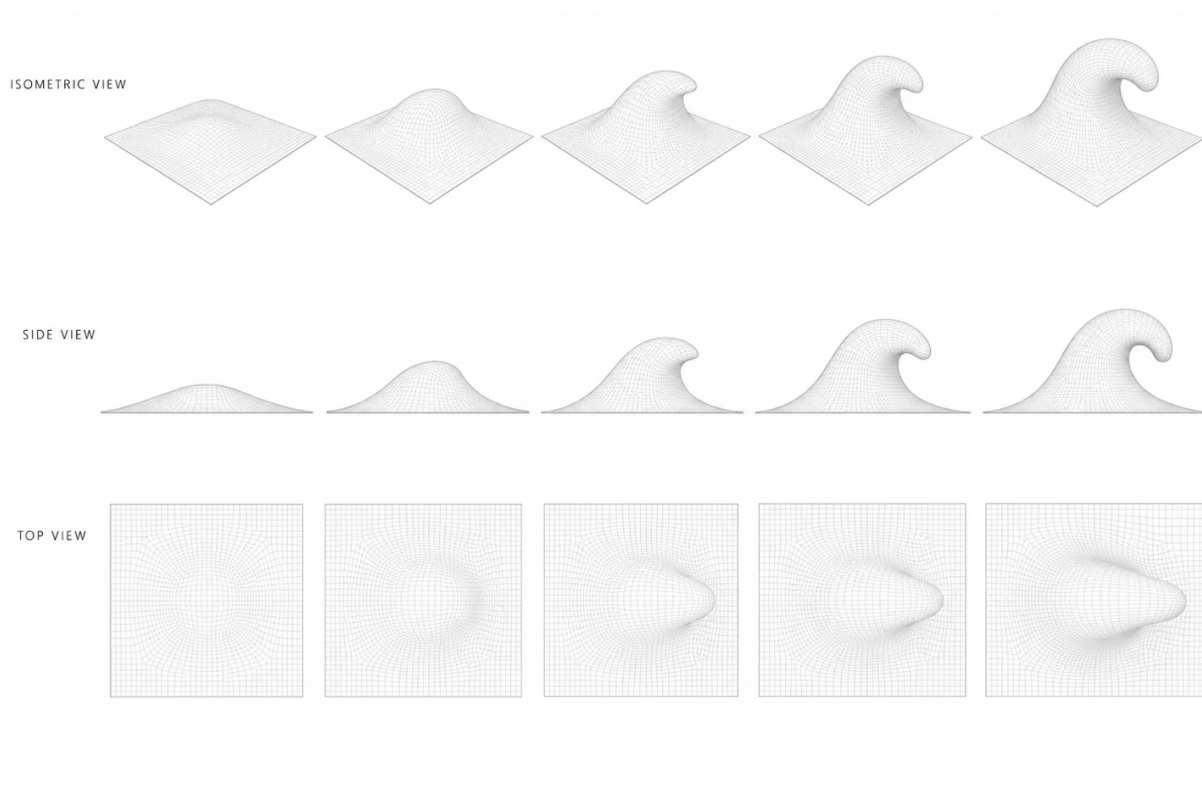
120

## 7 Accepted and Rejected Evidence

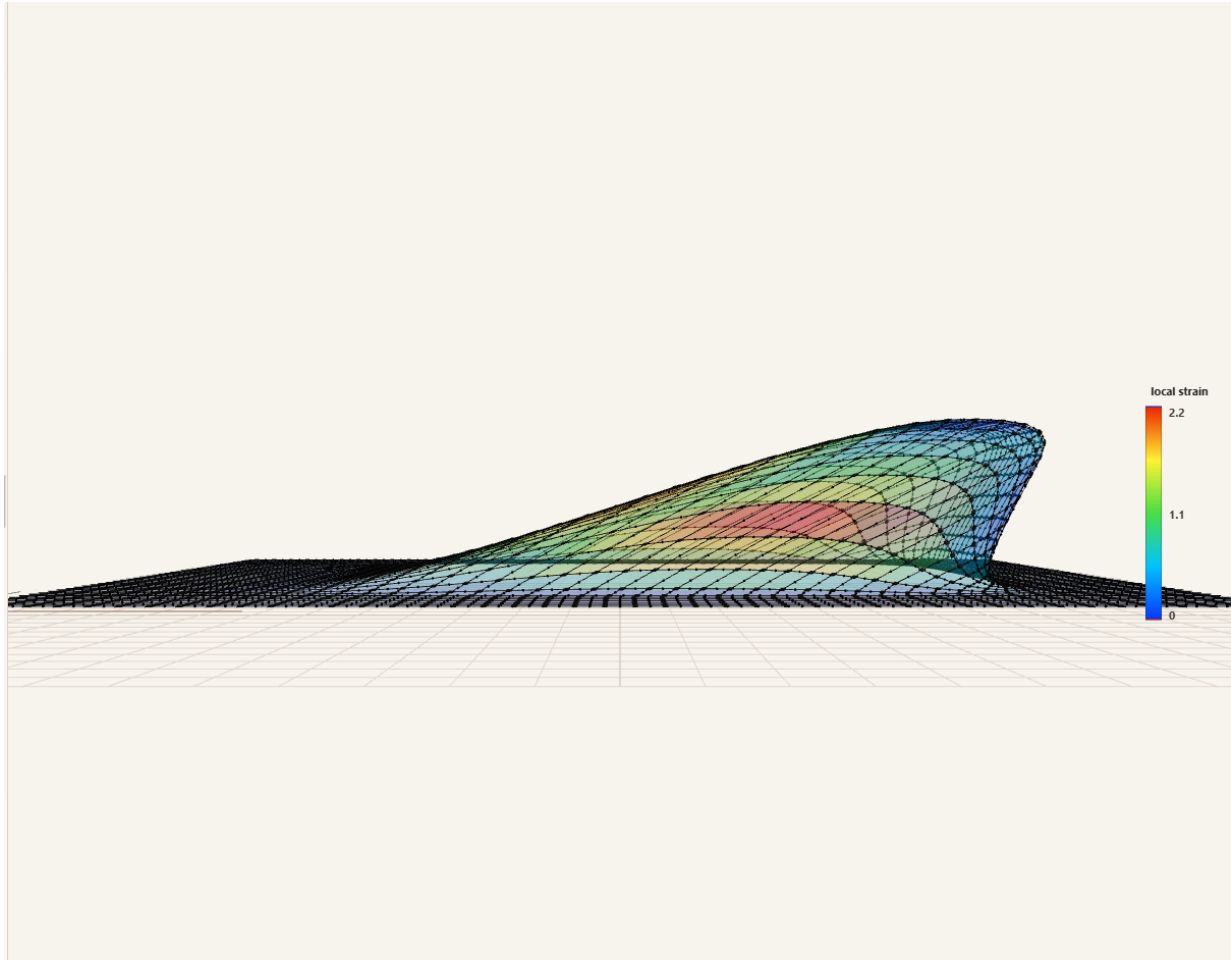
121



**Figure 1.** Accepted visual reference family: smooth gridded breaking-wave overview. This is a target family for the rendered sheet, not evidence that the current lattice already fabricates the shape.



**Figure 2.** Accepted reference sequence. The sequence records the desired global read: flat sheet, rising face, curling lip, open throat, and no dark pocket under the overhang.



**Figure 3.** Current simulator evidence figure used as a local output check. This figure is evidence only for the rendered state at the time it was exported; it is not a fabrication proof.

Rejected evidence includes old candidate screenshots that looked like tunnels, side walls, half-pipes, bowls, pointy spears, broad blobs, or hidden-mechanism silhouettes. Those images can remain in a private failure ledger, but they should not appear as authoritative paper figures.

## 8 Validation Gates

The inverse-sheet paper requires these gates before the public app or paper can claim the target field is current:

1. **Boundary gate:** boundary nodes stay flat and fixed.
2. **Zero gate:** morph  $m = 0$ , zero height, and zero offset produce near-zero metrics.
3. **Resolution gate:** minimum and default grids build the expected node, edge, quad, and dihedral

counts.

4. **Control gate:** height, overhang width, overhang angle, smoothing, flat contribution, and profile scale visibly change the intended part of the shape.

5. **Blocker gate:** known dimple, bowl, pocket, pointy-profile, and hidden-mechanism classes are rejected before paper or deployment claims.

6. **Visual gate:** desktop and mobile renders show the full sheet, not only a surface trace.

The current source scripts cover part of this gate set. They are necessary but not sufficient because Alan's prior corrections showed that automated metrics can pass while the visible sheet still looks wrong.

## 9 Limitations

The inverse-sheet tab does not solve material mechanics. It has no constitutive law, plate thickness, hinge stiffness, pneumatic pressure, contact, friction, self-collision, actuator force, or fabrication tolerance. It is closer to a source-bounded inverse-geometry workbench than to COMSOL, Abaqus, SOFA, or a measured physical build. The paper must therefore use bounded language:

- allowed: target deformation, geometric strain proxy, bend proxy, shear proxy, planarity proxy, known blocker, candidate comparison;
- not allowed: validated stress, validated force, measured feasibility, printable mechanism proof, physical prototype success, or exact reference match.

## 10 Materials and Methods

All calculations are client-side TypeScript. The active source is `src/latticeGeometry.ts`. The rest grid is built from sanitized configuration values. The target grid is computed through `targetFromDeformationField`, smoothed through named passes, and then measured through edge, node, quad, and dihedral metric functions. The UI exports metrics CSV and config JSON so an accepted candidate can be reproduced.

## 11 Data availability

The reference figures and generated proof figures are stored in `proofs/references/` and `proofs/figures/`. The current implementation files are in `src/`. No physical measurement dataset is included in this paper.

## 12 Code availability

The active code is in the WaveVis repository under `C:/Users/phama/Documents/gptshit/_apps/wavevis.aolabs.io`. Public bundles are served through the WaveVis deployment and the AO Labs mirror when deployed.

## 13 Additional information

This paper should be read with the double-layer Sarrus paper and the magnetic valve paper. The inverse-sheet paper defines the target deformation and geometric metrics. The Sarrus paper defines the mechanism constraint branch and the reason some visually plausible X-cell layouts fail. The magnetic valve paper defines the separate EPM/ball-pull sandbox and its visualization limits.

## References

1. D. Terzopoulos, J. Platt, A. Barr, K. Fleischer, Elastically Deformable Models. *ACM SIGGRAPH Computer Graphics* **21**, 205–214 (1987).
2. A. Witkin, M. Kass, presented at the Proceedings of SIGGRAPH 1988, pp. 159–168.
3. E. Grinspun, A. N. Hirani, M. Desbrun, P. Schröder, presented at the Proceedings of the 2003 ACM SIGGRAPH/Eurographics Symposium on Computer Animation, pp. 62–67.
4. S. Bouaziz, S. Martin, T. Liu, L. Kavan, M. Pauly, Projective Dynamics: Fusing Constraint Projections for Fast Simulation. *ACM Transactions on Graphics* **33**, 154:1–154:11 (2014).
5. T. Liu, A. W. Bargteil, J. F. O’Brien, L. Kavan, Fast Simulation of Mass-Spring Systems. *ACM Transactions on Graphics* **32**, 214:1–214:7 (2013).
6. T. Du *et al.*, DiffPD: Differentiable Projective Dynamics. *ACM Transactions on Graphics* **41**, 13:1–13:21 (2022).

- 
- 180 7. M. Skouras *et al.*, Designing Inflatable Structures. *ACM Transactions on Graphics* **33**, 63:1–63:10  
181 (2014).
- 182 8. J. Panetta *et al.*, Computational Inverse Design of Surface-Based Inflatables. *ACM Transactions*  
183 *on Graphics* **40**, 40:1–40:14 (2021).
- 184 9. M. Konaković-Luković *et al.*, Beyond Developable: Computational Design and Fabrication with  
185 Auxetic Materials. *ACM Transactions on Graphics* **35**, 89:1–89:11 (2016).
- 186 10. M. Schenk, S. D. Guest, Geometry of Miura-Folded Metamaterials. *Proceedings of the National*  
187 *Academy of Sciences* **110**, 3276–3281 (2013).
- 188 11. J. L. Silverberg *et al.*, Using Origami Design Principles to Fold Reprogrammable Mechanical  
189 Metamaterials. *Science* **345**, 647–650 (2014).
- 190 12. E. T. Filipov, T. Tachi, G. H. Paulino, Origami Tubes Assembled into Stiff, Yet Reconfigurable  
191 Structures and Metamaterials. *Proceedings of the National Academy of Sciences* **112**, 12321–  
192 12326 (2015).
- 193 13. C. Coulais, E. Teomy, K. de Reus, Y. Shokef, M. van Hecke, Combinatorial Design of Textured  
194 Mechanical Metamaterials. *Nature* **535**, 529–532 (2016).